

CSC3209 Software architecture and Design Patterns

Lecture 5: Understanding Quality Attributes

Subject Lecturer:
Dr. Muhammed Basheer Jasser

References

- Materials from
 - Chapter 4: Len Bass, Paul Clements, Rick Kazman, Software Architecture in Practice, 3/E

Lecture Outline

- Architecture and Requirements
- Functionality
- Quality Attribute Considerations
- Specifying Quality Attribute Requirements
- Achieving Quality Attributes through Tactics
- Guiding Quality Design Decisions
- Summary

Architecture and Requirements

- System requirements can be categorized as:
 - Functional requirements. These requirements state what the system must do, how it must behave or react to run-time stimuli.
 - Quality attribute requirements. These requirements annotate (qualify) functional requirements. Qualification might be how fast the function must be performed, how resilient it must be to erroneous input, how easy the function is to learn, etc.
 - Constraints. A constraint is a design decision with zero degrees of freedom. That is, it's a design decision that has already been made for you.
 - Examples include the requirement to use a certain programming language or to reuse a certain existing module.

Functionality

- Functionality is the ability of the system to do the work for which it was intended.
- Functionality has a strange relationship to architecture:
 - Functionality does not determine architecture; given a set of required functionality, there is no end to the architectures you could create to satisfy that functionality
 - At the very least, you could divide up the functionality in any number of ways and assign the sub pieces to different architectural elements.
 - In fact, if functionality were the only thing that mattered, you wouldn't have to divide the system into pieces at all

Functionality

- Instead, we design our systems as structured sets of cooperating architectural elements—modules, layers, classes, services, databases, apps, threads, peers, tiers, and on and on—to make them understandable and to support a variety of other purposes.
- Those “other purposes” are the other quality attributes
- Functionality and quality attributes are orthogonal.

Quality Attribute Considerations

- If a **functional requirement** is "when the user presses the green button the Options dialog appears":
 - a **performance QA** annotation might describe **how quickly** the dialog will appear;
 - an **availability QA** annotation might describe **how often** this **function** will **fail**, and **how quickly** it will be **repaired**;
 - a **usability QA** annotation might describe **how easy** it is to **learn** this function.

Quality Attribute Considerations

- From an architect's perspective, there are three problems with previous discussions of quality attributes:
 1. The definitions provided for an attribute are not testable. It is meaningless to say that a system will be "modifiable".
 - Every system may be modifiable with respect to one set of changes and not modifiable with respect to another.
 - The other quality attributes are similar in this regard: a system may be robust with respect to some faults and brittle with respect to others.

Quality Attribute Considerations

- From an architect's perspective, there are three problems with previous discussions of quality attributes:
2. Endless time is wasted on arguing over which quality a concern belongs to. Is a system failure due to a denial of service attack an aspect of availability, performance, security, or usability?
 - All four attribute communities would claim ownership of a system failure due to a denial-of-service attack. All are, to some extent, correct.
 - But this doesn't help us, as architects, understand and create architectural solutions to manage the attributes of concern.

Quality Attribute Considerations

- From an architect's perspective, there are three problems with previous discussions of quality attributes:
3. Each attribute community has developed its own vocabulary.
 - The performance community has “events” arriving at a system,
 - the security community has “attacks” arriving at a system,
 - the availability community has “failures” of a system,
 - and the usability community has “user input.”
 - All of these may actually refer to the same occurrence, but they are described using different terms.

Quality Attribute Considerations

- A **solution** to the first two of these problems (**untestable definitions** and **overlapping concerns**) is to use *quality attribute scenarios* as a means of characterizing quality attributes.
- A **solution** to the **third problem (different communities vocabularies)** is to **provide a discussion of each attribute**—concentrating on its underlying concerns—to **illustrate the concepts** that are **fundamental to that attribute community**.

Quality Attribute Considerations

- There are **two categories** of **quality attributes** on which we focus.
- The **first** is those that describe some **property** of the **system** at **runtime**, such as **availability**, **performance**, or **usability**.
- The **second** is those that **describe** some **property** of the **development** of the **system**, such as **modifiability** or **testability**.

Quality Attribute Considerations

- Within complex systems, **quality attributes** can **never** be **achieved** in **isolation**.
- The **achievement** of any **one** will have an **effect**, sometimes **positive** and sometimes **negative**, **on** the achievement of **others**.
- For example, **almost every quality attribute** negatively **affects performance**.
 - Take **portability**. The main technique for **achieving portable** software is to **isolate** system **dependencies**, which **introduces overhead** into the system's execution, typically as process or procedure boundaries, and this hurts performance.

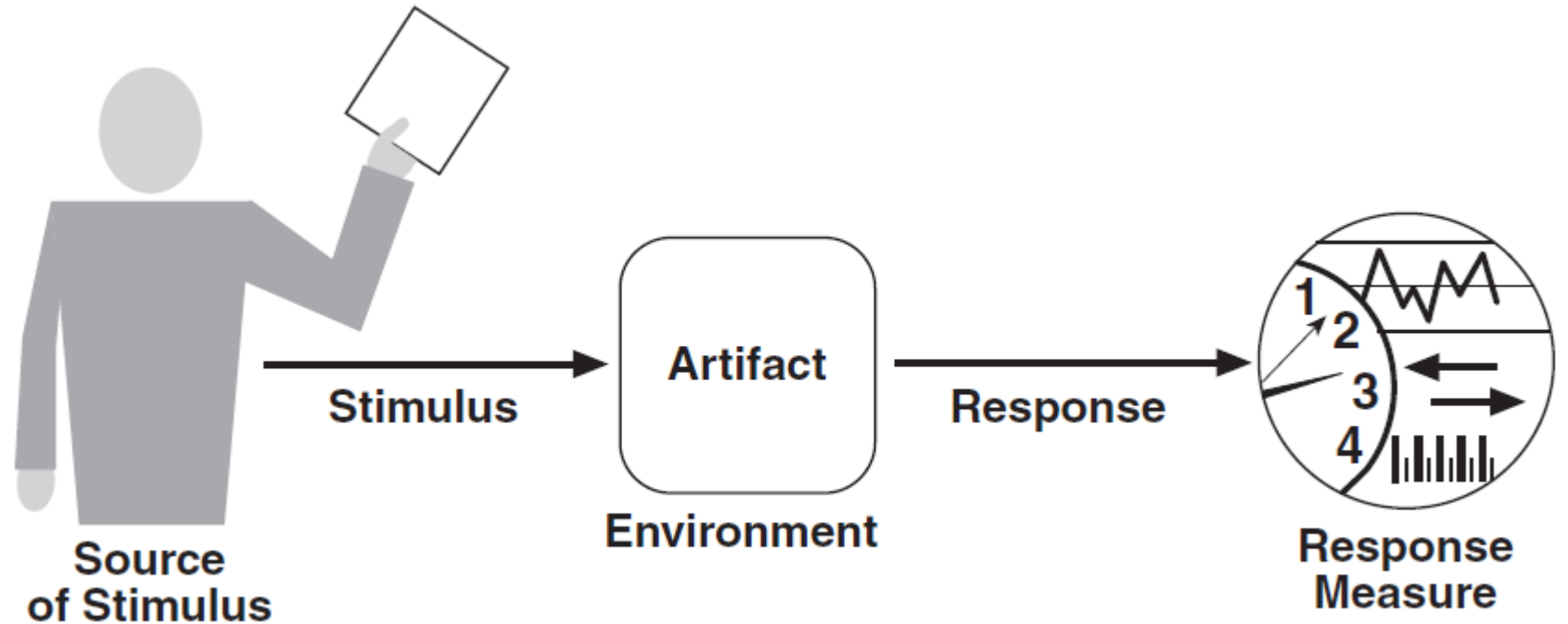
Quality Attribute Considerations

- Determining the design that satisfies all of the quality attribute requirements is partially a matter of making the appropriate tradeoffs.
- Our purpose here is to provide the context for discussing each quality attribute. In particular, we focus on:
 - how quality attributes can be specified,
 - what architectural decisions will enable the achievement of particular quality attributes,
 - what questions about quality attributes will enable the architect to make the correct design decisions.

Specifying Quality Attribute Requirements

- A **quality attribute requirement** should be **unambiguous** and **testable**
- We **use a common form** to **specify** all **quality attribute requirements** as **scenarios**.
- Our representation of **quality attribute scenarios** has these **parts**:

1. **Stimulus**
2. **Stimulus source**
3. **Response**
4. **Response measure**
5. **Environment**
6. **Artifact**



Specifying Quality Attribute Requirements

- 1. Stimulus.** The stimulus is a **condition/event** that **requires a response** when it arrives at a system. The stimulus can be:
 - an **event** to the **performance community**, a **user operation** to the **usability community**, or an **attack** to the **security community**.
- 2. Source of stimulus.** This is some **entity** (a human, a computer system, or any other actuator) that **generated the stimulus**.
 - The **source** of the **stimulus** may affect how it is **treated** by the system. A **request** from a **trusted user** will **not undergo** the **same scrutiny** as a **request** by an **untrusted user**.

Specifying Quality Attribute Requirements

3. Response. The response is the activity undertaken as the result of the arrival of the stimulus.

- The response consists of the responsibilities that the system (for runtime qualities) or the developers (for development-time qualities) should perform in response to the stimulus.
- For example, in a performance scenario, an event arrives (the stimulus) and the system should process that event and generate a response.
- In a modifiability scenario, a request for a modification arrives (the stimulus) and the developers should implement the modification—without side effects—and then test and deploy the modification.

4. Response measure. When the response occurs, it should be measurable in some fashion so that the requirement can be tested.

- Determining whether a response is satisfactory—whether the requirement is satisfied—is enabled by providing a response measure.
- For performance this could be a measure of latency or throughput;
- for modifiability it could be the labor or wall clock time required to make, test, and deploy the modification.

Specifying Quality Attribute Requirements

- These **four characteristics** of a scenario are the **heart** of our **quality attribute specifications**. But there are **two** more **characteristics** that are **important**: **environment** and **artifact**.

5. Environment. the set of **circumstances** in which the **scenario takes place**. The **stimulus occurs under** certain **conditions**. The system may be in an **overload** condition or in **normal** operation, or some other relevant state.

- For example, a failure that is the **fifth** successive **failure** of a component may be **treated differently** than the **first failure** of that component.

6. Artifact. the **portion of the system** to which the **requirement applies**. This may be the **whole system**, or some **piece** or pieces of it.

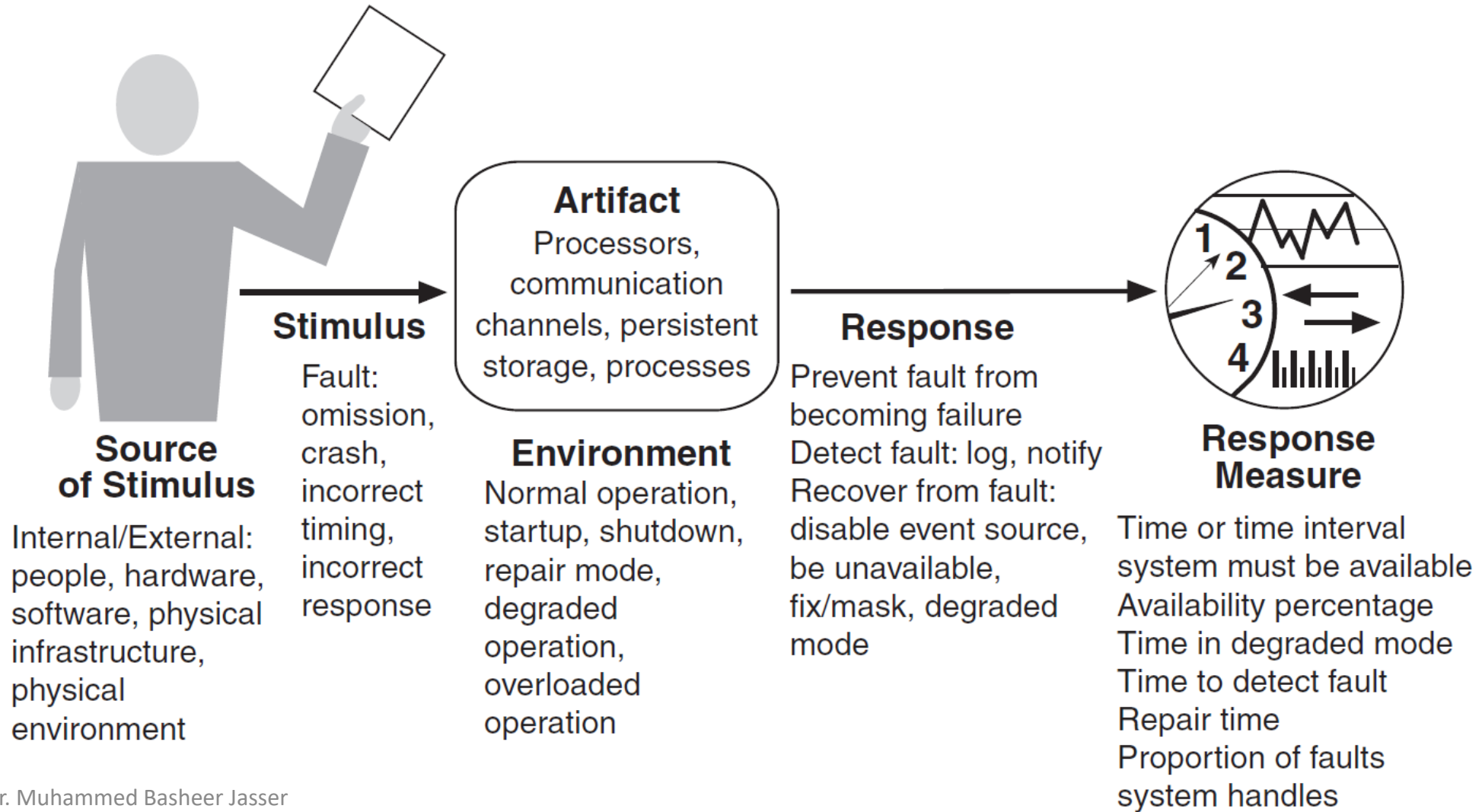
- **Modifications to the user interface** may have **faster response** times **than modifications** to the **middleware**.

Specifying Quality Attribute Requirements

- We distinguish:
 - *general quality attribute scenarios* (“general scenarios”)—those that are system independent and can, potentially, pertain to *any system*
 - from *concrete quality attribute scenarios* (concrete scenarios)—those that are specific to the *particular system* under consideration.
- We *can characterize quality attributes* as a *collection* of *general scenarios*.
 - Of course, *to translate* these *generic attribute characterizations* into *requirements for a particular system*, the *general scenarios* need to be *made system specific*.

Specifying Quality Attribute Requirements

- Example **general scenario** for **availability**:



Achieving Quality Attributes Through Tactics

- There are a collection of **primitive design techniques** that an architect can use to **achieve** a **quality attribute response**.
- We call these architectural design primitives *tactics*.
- Tactics, like design patterns, are **techniques** that **architects** have been **using** for **years**. We do not *invent* tactics, we simply capture what architects do in **practice**.

Achieving Quality Attributes Through Tactics

- **Why** do we do this? There are three reasons:
 1. **Design patterns are complex**; they are a bundle of design decisions. But patterns are often difficult to apply as is; **architects** need to **modify** and **adapt them**. **By** understanding **tactics**, an architect can **assess** the **options** for **augmenting** an **existing pattern** to achieve a quality attribute goal.
 2. **If no pattern exists** to **realize** the architect's **design goal**, **tactics allow** the architect to **construct** a **design** fragment from “**first principles**”.
 3. **By cataloguing tactics**, we **make design more systematic**. You frequently will have a choice of **multiple tactics** to **improve** a **particular quality attribute**. The choice of which tactic to use depends on factors such as tradeoffs among other quality attributes and the cost to implement.

Guiding Quality Design Decisions

- Architecture design is a systematic approach to making design decisions.
- We categorize the design decisions that an architect needs to make as follows:
 1. Allocation of responsibilities
 2. Coordination model
 3. Data model
 4. Management of resources
 5. Mapping among architectural elements
 6. Binding time decisions
 7. Choice of technology
- These categories are not the only way to classify architectural design decisions, but they do provide a rational division of concerns.
- These categories might overlap, but it's all right if a particular decision exists in two different categories, because the concern of the architect is to ensure that every important decision is considered.

Allocation of Responsibilities

- Decisions involving allocation of responsibilities include:
 - identifying the important responsibilities including basic system functions, architectural infrastructure, and satisfaction of quality attributes.
 - determining how these responsibilities are allocated to non-runtime and runtime elements (namely, modules, components, and connectors).
- Strategies for making these decisions include functional decomposition, modeling real-world objects, grouping based on the major modes of system operation, or grouping based on similar quality requirements: processing frame rate, security level, or expected changes.

Coordination Model

- Software works by having **elements interact** with each other **through designed mechanisms**. These mechanisms are collectively **referred to** as a **coordination model**.
- Decisions about the coordination model include:
 - **identify** the **elements** of the system that **must coordinate**, or are **prohibited** from coordinating
 - **determining** the **properties** of the **coordination**, such as timeliness, currency, completeness, **correctness**, and **consistency**
 - **choosing** the **communication mechanisms** that **realize those properties**. Important properties of the communication mechanisms include stateful vs. stateless, **synchronous vs. asynchronous**, guaranteed vs. non-guaranteed delivery, and performance-related properties such as throughput and latency

Data Model

- Every system must represent artifacts of system-wide interest—data—in some internal fashion. The collection of those representations and how to interpret them is referred to as the data model.
- Decisions about the data model include:
 - choosing the major data abstractions, their operations, and their properties. This includes determining how the data items are created, initialized, accessed, persisted, manipulated, translated, and destroyed.
 - metadata needed for consistent interpretation of the data
 - organization of the data. This includes determining whether the data is going to be kept in a relational data base, a collection of objects or both

Management of Resources

- An architect **needs** to **arbitrate** the **use** of **shared resources** in the **architecture**. These include **hard resources** (e.g., CPU, memory, battery, hardware buffers, system clock, I/O ports) and **soft resources** (e.g., system locks, software buffers, thread pools, and non-thread-safe code).
- **Decisions** for management of resources include:
 - **identifying** the **resources** that must be managed and determining the **limits** for each
 - **determining which system element(s) manage** each **resource**
 - **determining how resources** are **shared** and the arbitration strategies employed when there is contention
 - **determining the impact** of **saturation** on different **resources**.
 - For example, as a **CPU** becomes more **heavily loaded**, **performance** usually just **degrades** fairly **steadily**.

Mapping Among Architectural Elements

- An **architecture** must **provide two types of mappings**.
 - **First**, there is **mapping between elements in different types of architecture structures**—for example, **mapping from units of development (modules) to units of execution** (threads or processes).
 - **Second**, there is **mapping between software elements and environment elements**—for example, **mapping from processes to the specific CPUs** where these processes will execute.
- Useful mappings include:
 - the **mapping** of **modules** and **runtime elements** to each other—that is, the **runtime elements** that are **created from each module**; the modules that contain the code for each runtime element
 - the **assignment** of **runtime elements** to **processors**
 - the **assignment** of **items** in the **data** model to **data stores**

Binding Time

- The decisions in the other categories have an associated binding time decision. Examples of such binding time decisions include:
 - For choice of **coordination model** you can design **run-time negotiation** of **protocols**.
 - For **resource management** you can design a system to **accept new peripheral** devices **plugged in** at **run-time**.

Choice of Technology

- Choice of technology decisions involve:
 - **deciding** which **technologies** are **available** to realize the decisions made in the other categories
 - **determining** whether the **tools** to support this technology (IDEs, simulators, testing tools, etc.) are **adequate**
 - **determining** the extent of internal **familiarity** and **external support** for the technology (such as courses, tutorials, examples, availability of contractors)
 - **determining** the **side effects** of **choosing** a **technology** such as a required coordination model or constrained resource management opportunities
 - **determining** whether a **new** technology is **compatible** with the **existing** technology stack

Summary

- **Requirements** for a system come in three categories.
 1. **Functional**. These requirements are satisfied by including an appropriate set of responsibilities within the design.
 2. **Quality attribute**. These requirements are satisfied by the structures and behaviors of the architecture.
 3. **Constraints**. A constraint is a design decision that's already been made.
- To express a quality attribute requirement we use a **quality attribute scenario**. The parts of the scenario are:
 1. **Source of stimulus**.
 2. **Stimulus**
 3. **Response**.
 4. **Response measure**.
 5. **Environment**.
 6. **Artifact**.

Summary

- An architectural tactic is a design decision that affects a quality attribute response. The focus of a tactic is on a single quality attribute response.
- Architectural patterns can be seen as “packages” of tactics.
- The seven categories of architectural design decisions are:
 1. Allocation of responsibilities
 2. Coordination model
 3. Data model
 4. Management of resources
 5. Mapping among architectural elements
 6. Binding time decisions
 7. Choice of technology